

Rapport Projet - Programmation répartie

Alexis CERDA DE ALMEIDA VILACA

Logan SAGET

Taïno HERMANN

Emelyne FOUSSE

9 juin 2026

Table des matières

1	Introduction	2
2	Le raytracer local	3
2.1	Mesures de temps d'exécution	3
2.2	Reproduction de l'image	4
3	Architecture de l'application répartie	6
3.1	Schéma de l'architecture	6
3.2	Processus fixes et processus mobiles	7
3.3	Données échangées entre les processus	7
4	Fonctionnement des composants	8
4.1	Le service central	8
4.2	Les nœuds de calcul	8
4.3	Le client / dispatcher	8
5	Mise en œuvre du parallélisme	9
5.1	Distribution des tâches en threads	9
5.2	Synchronisation et assemblage des résultats	9
6	Conclusion	10

1 Introduction

Lien du répertoire GIT : https://github.com/HermannT88/projet_rmi_calcul_parallele

Pour

2 Le raytracer local

2.1 Mesures de temps d'exécution

Pour réaliser des mesures sur le temps de calcul, nous avons fait une boucle qui exécute 10 fois le calcul de l'image avec à chaque itération `largeur = i * largeur de base` et `hauteur = i * hauteur de base`

```
for (int i = 1; i <= 10; i++) {  
    int l = largeur*i, h = hauteur*i;  
  
    Instant debut = Instant.now();  
    System.out.println("Itération " + i + " Largeur = " + l + " Hauteur = " + h);  
    Image image = scene.compute(x0, y0, l, h);  
    Instant fin = Instant.now();  
  
    long duree = Duration.between(debut, fin).toMillis();  
  
    System.out.println("Temps de calcul : "+duree+" ms");  
}
```

Nous obtenons alors ces résultats :

Itération	Largeur (px)	Hauteur (px)	Temps de calcul (ms)
1	512	512	3182
2	1024	1024	7372
3	1536	1536	13642
4	2048	2048	23977
5	2560	2560	32773
6	3072	3072	48813
7	3584	3584	64370
8	4096	4096	84049
9	4608	4608	107256
10	5120	5120	128906

TABLE 1 – Évolution du temps de calcul en fonction des dimensions

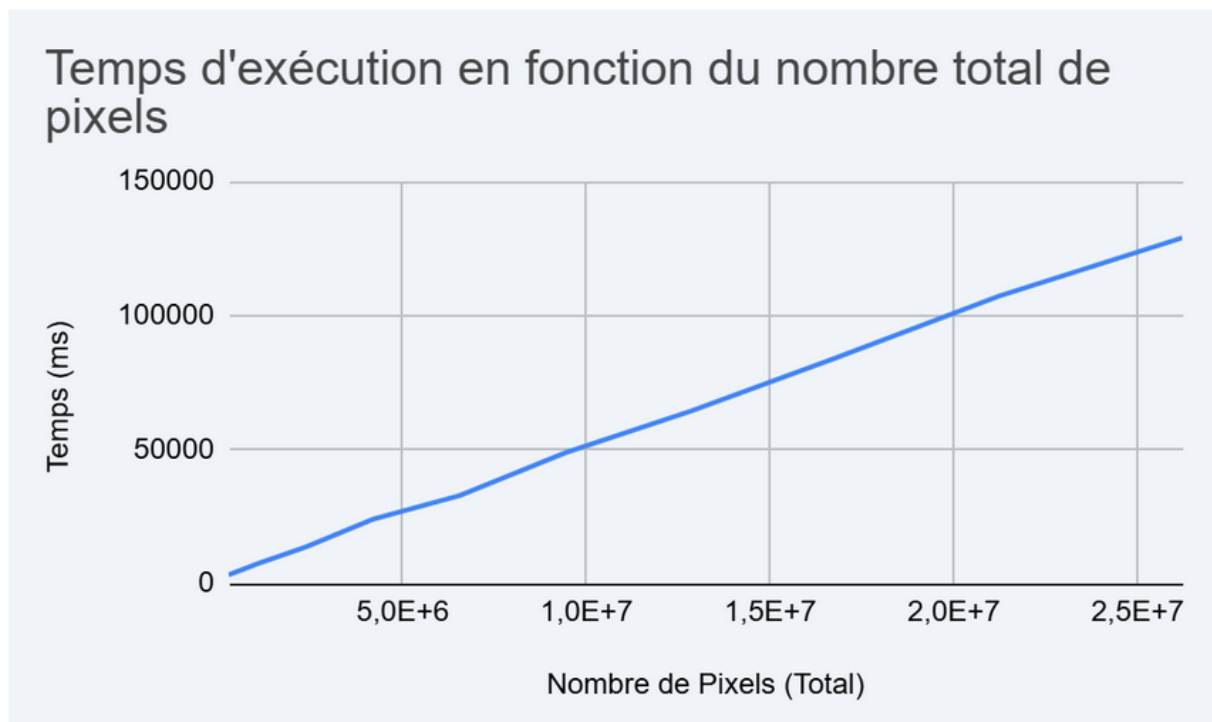


FIGURE 1 – Temps d'exécution en fonction des pixels

Nous constatons que le temps de calcul augmente proportionnellement au nombre de pixels de l'image.

2.2 Reproduction de l'image

L'objectif est de reproduire l'image ci-dessous :

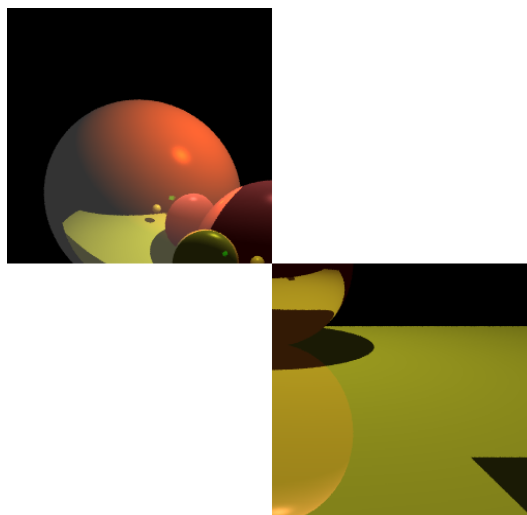


FIGURE 2 – Image à reproduire

Pour reproduire l'image, nous allons calculer individuellement les deux parties de l'image attendue.

La première partie, le programme va calculer l'image en partant des coordonnées 0 ; 0 soit le coin supérieur droit. Puis il va calculer l'image jusqu'à son centre, donc à la moitié de la hauteur et la moitié de la largeur. Ce qui nous donne la ligne suivante :

```
Image image1 = scene.compute(x0, y0, l/2, h/2);
```

Pour la seconde, nous allons recréer une image qui cette fois va commencer au milieu et encore avancé en largeur et en hauteur de la moitié. Nous avons alors cette ligne :

```
Image image2 = scene.compute(l/2, h/2, l/2, h/2);
```

Enfin nous affichons les deux images calculées précédemment sur la même fenêtre, avec la première au coordonnées 0 ; 0 et la seconde au centre de l'image :

```
disp.setImage(image1, x0, y0);  
disp.setImage(image2, l/2, h/2);
```

Pour le code complet, nous arrivons donc à ce résultat :

```
int x0 = 0, y0 = 0;  
int l = largeur, h = hauteur;  
  
// Chronométrage du temps de calcul  
Instant debut = Instant.now();  
System.out.println("Calcul de l'image :\n - Coordonnées : "+x0+", "+y0  
+" \n - Taille "+ largeur + "x" + hauteur);  
Image image1 = scene.compute(x0, y0, l/2, h/2);  
Image image2 = scene.compute(l/2, h/2, l/2, h/2);  
Instant fin = Instant.now();  
  
long duree = Duration.between(debut, fin).toMillis();  
  
System.out.println("Image calculée en : "+duree+" ms");  
  
// Affichage de l'image calculée  
disp.setImage(image1, x0, y0);  
disp.setImage(image2, l/2, h/2);
```

3 Architecture de l'application répartie

3.1 Schéma de l'architecture

Pour notre première idée d'architecture, l'application avait un client qui envoyait le fichier txt correspondant à l'image à calculer au service central. Ce service se serait ensuite chargé du découpage en threads et de la répartition entre les nœuds de calcul. Une fois l'image calculée entièrement, le service central aurait renvoyé l'objet Image au Client.

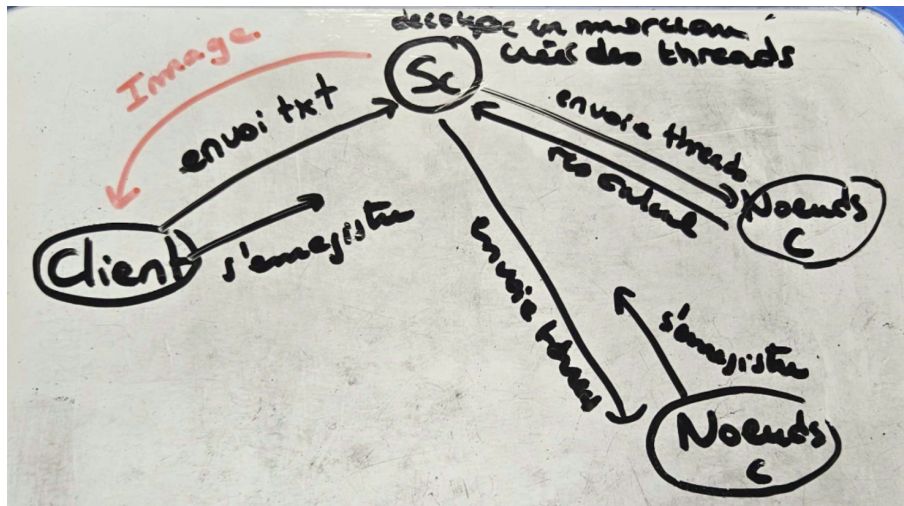


FIGURE 3 – Première version

Pour la deuxième version, le fonctionnement était très similaire à la première version, avec cette fois toute la partie découpage des tâches et échanges avec les nœuds déléguée à un service Répartiteur.

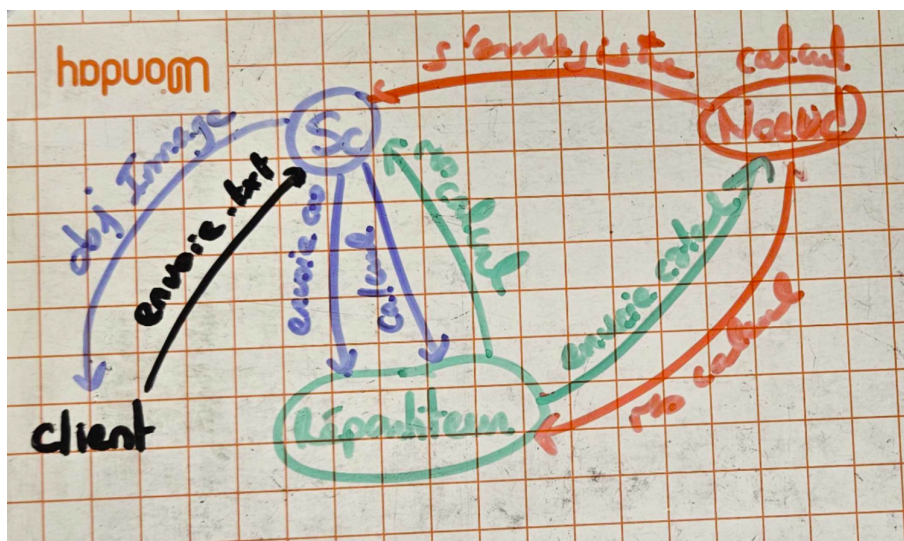


FIGURE 4 – Deuxième version

Dans la version finale, toute la logique de découpage en threads a été redonnée au Client. Le service central ne sert plus qu'à fournir et gérer les nœuds, c'est-à-dire maintenir une liste de

nœuds, en ajouter, en supprimer et envoyer un nœud lorsqu'un Client le demande.

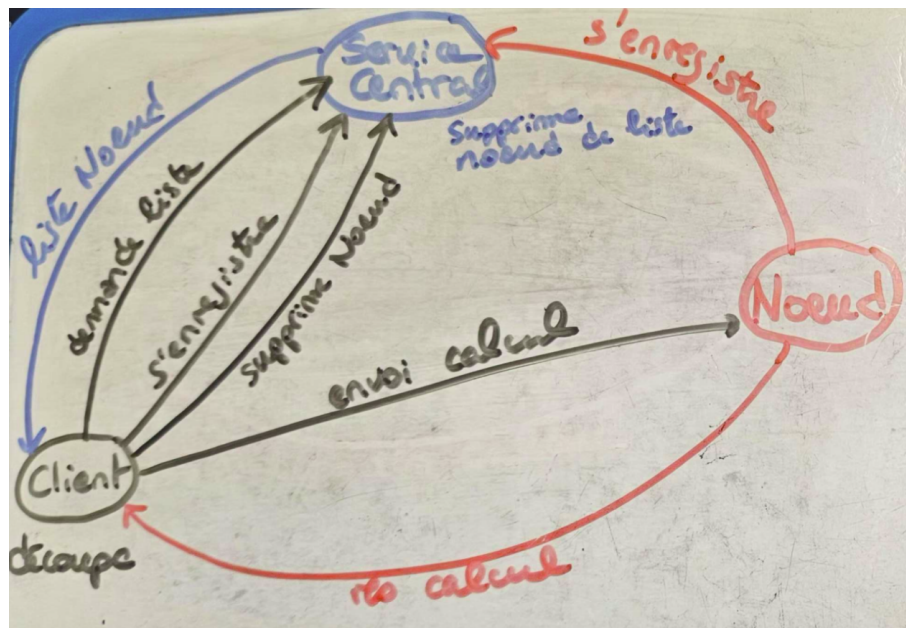


FIGURE 5 – Version finale

* Contrairement à ce qui est indiqué sur le schéma, le service central renvoie un unique nœud au client et non la liste entière.

3.2 Processus fixes et processus mobiles

Les processus mobiles sont représentés par les différents nœuds de calculs qui peuvent s'enregistrer et se déconnecter. On peut également considérer le client qui est à l'origine de la demande de calcul comme un processus mobile.

Le processus fixe est le service qui va transmettre le calcul aux différents nœuds de calculs. Le service est enregistré sur un port.

3.3 Données échangées entre les processus

Le client envoie un fichier .txt et les coordonnées relatives à la création de l'image (deux pour le point initial et deux pour la longueur et la largeur des pixels). Il reçoit un objet "Image" une fois celle-ci construite.

Une fois ces calculs réalisés, les nœuds retournent le résultat et le service central va l'interpréter.

4 Fonctionnement des composants

4.1 Le service central

4.2 Les nœuds de calcul

4.3 Le client / dispatcher

5 Mise en œuvre du parallélisme

5.1 Distribution des tâches en threads

5.2 Synchronisation et assemblage des résultats

6 Conclusion